CyberMinions

DevSecOps CI/CD Pipeline with Kubernetes Monitoring

By: Shreya Kumari, Zaid Khan, Saniya Gandhi

1. Abstract

This project demonstrates the implementation of a complete DevSecOps Continuous Integration and Continuous Deployment (CI/CD) pipeline for deploying a containerized web application on Kubernetes. The solution integrates source code management, automated code quality analysis, vulnerability assessment, containerization, image registry management, orchestration, monitoring, and visualization tools.

The project automates the software delivery lifecycle from source code commit to Kubernetes deployment while incorporating security scanning and real-time infrastructure monitoring using Prometheus and Grafana.

2. Project Objective

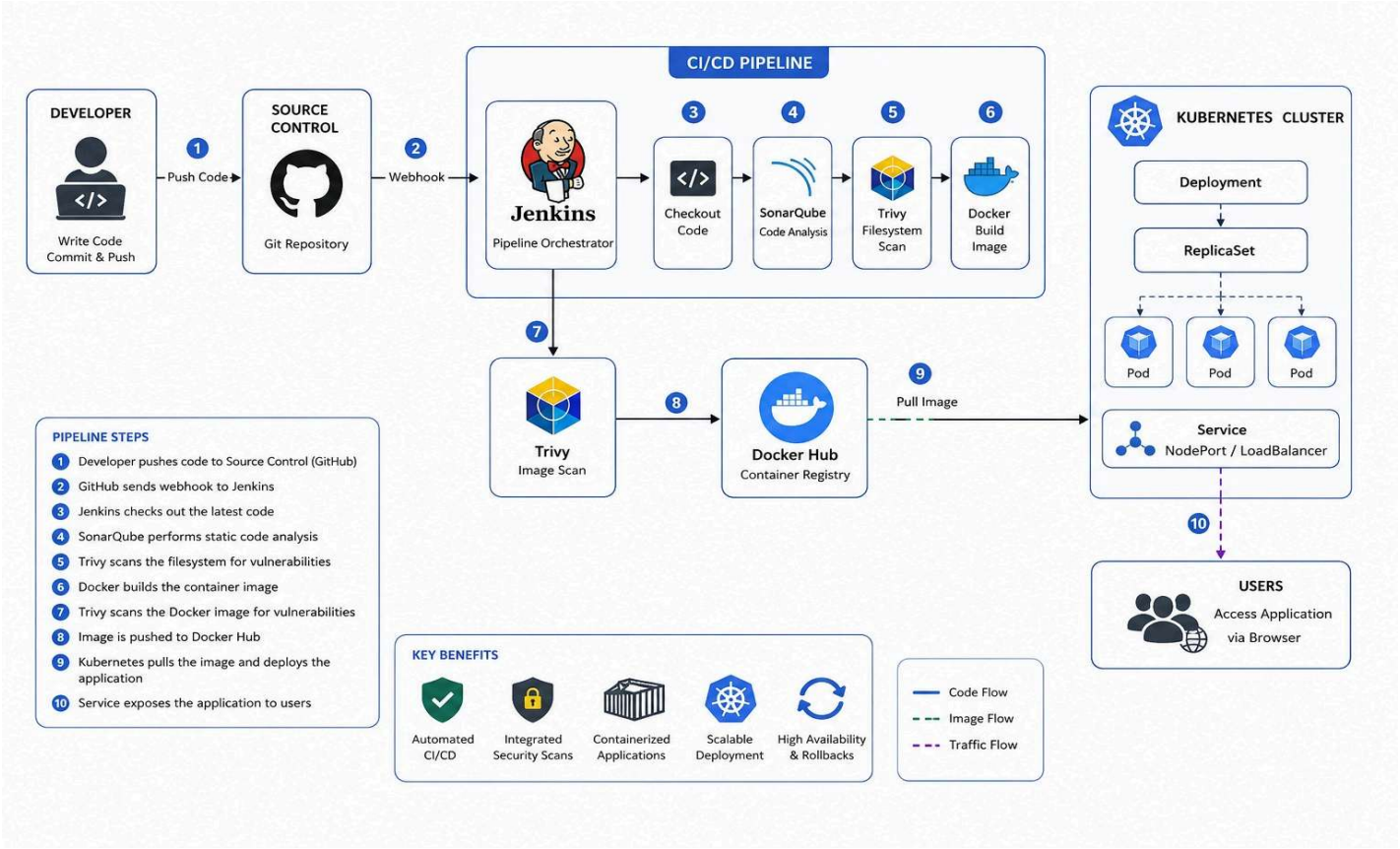
The primary objectives of this project are:

- Automate application build and deployment.
- Implement DevSecOps practices.
- Perform static code analysis using SonarQube.
- Identify vulnerabilities using Trivy.
- Containerize applications using Docker.
- Store container images in Docker Hub.
- Deploy applications on Kubernetes.
- Enable automated deployment through Jenkins.
- Monitor infrastructure and services using Prometheus and Grafana.

3. Technology Stack

Category	Tool
Version Control	GitHub
CI/CD	Jenkins
Code Quality	SonarQube
Security Scanning	Trivy
Containerization	Docker
Registry	Docker Hub
Orchestration	Kubernetes
Monitoring	Prometheus
Visualization	Grafana
Application	Static Web Application

4. Solution Architecture



5. Infrastructure Setup

Jenkins Server

Responsibilities:

- Pipeline orchestration
- SonarQube integration
- Trivy scanning
- Docker image creation
- Kubernetes deployment
- Installed Components
- Jenkins
- Docker
- Trivy
- Sonar Scanner
- kubectl
- Kubernetes Cluster

Master Node

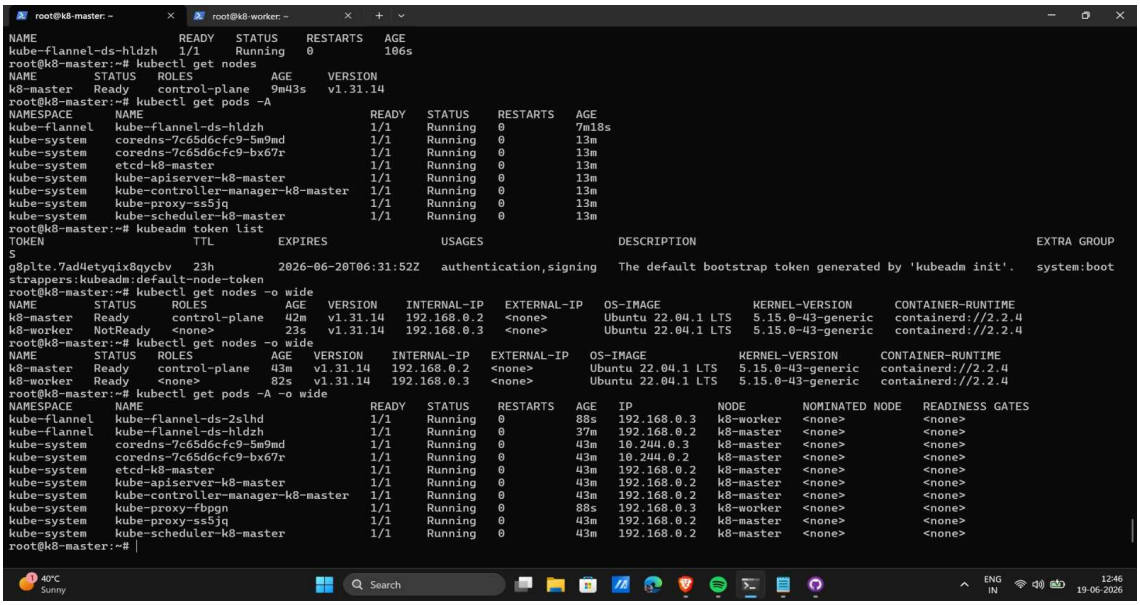
Responsibilities:

- Cluster management
- Scheduling
- Deployment management

Worker Node

Responsibilities:

- Running application workloads
- Hosting application pods

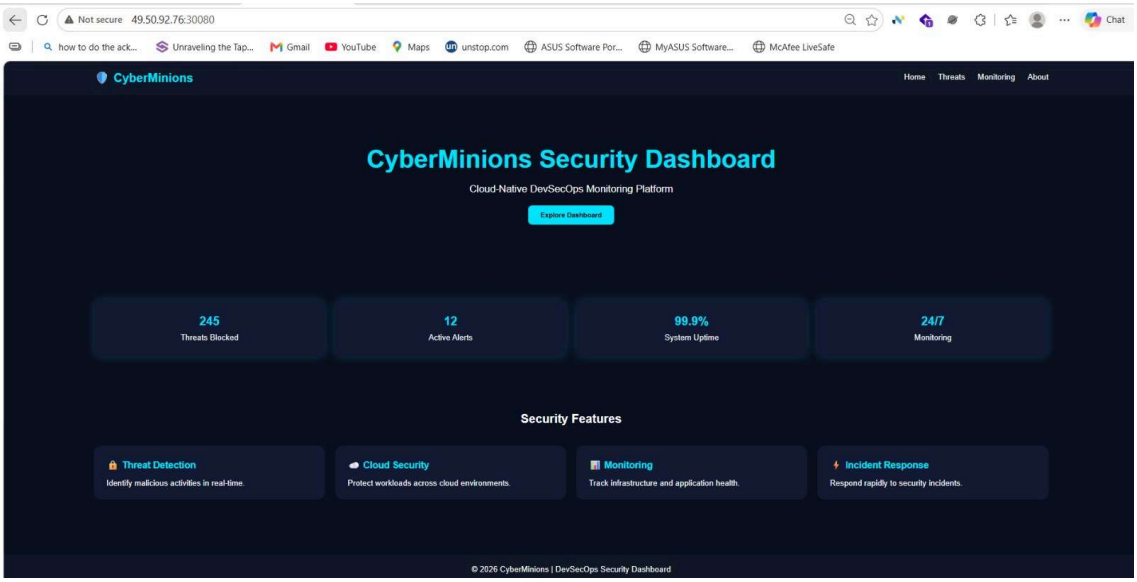


6. Application Development

A static web application was developed as the deployment target for the DevSecOps pipeline.
Application components:

- index.html
- style.css
- script.js
- Dockerfile

The application was stored in a GitHub repository and became the source for all CI/CD activities.

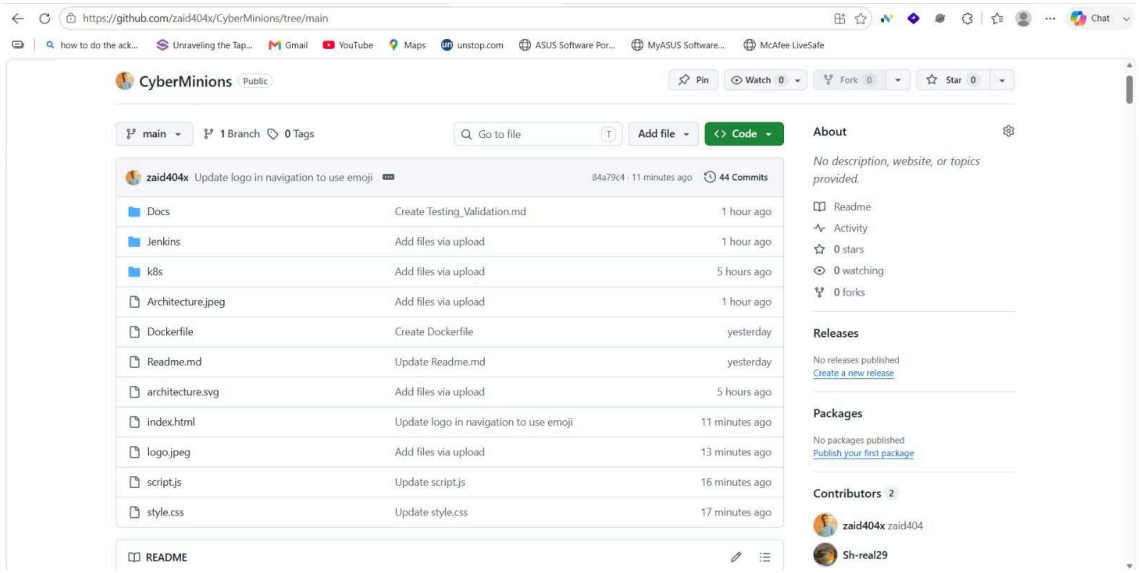


7. Source Code Management

GitHub was used as the centralized source control system.

- Activities Performed
- Repository creation
- Source code upload
- Jenkins integration
- Webhook configuration
- Benefits
- Version control
- Collaboration
- Automated build triggering

https://github.com/zaid404x/CyberMinions



8. Jenkins Pipeline Implementation

A Declarative Jenkins Pipeline was created to automate the entire software delivery process.

Pipeline Stages
Stage 1: Checkout
Jenkins retrieves the latest source code from GitHub.

Stage 2: SonarQube Analysis
Static code analysis is performed.
Checks include: Code quality, Bugs, Code smells, Maintainability

Stage 3: Trivy Filesystem Scan
The application source files are scanned for vulnerabilities.

Stage 4: Docker Build

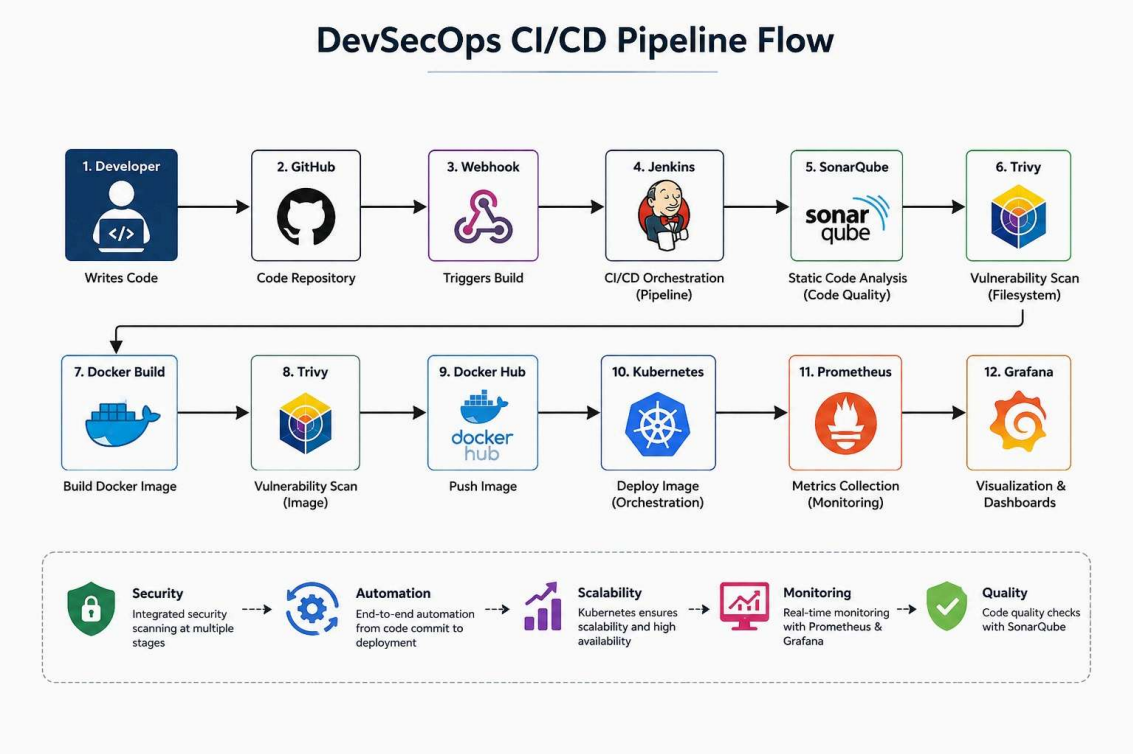
Docker image is created.
Example: docker build -t cyberminions:v1 .

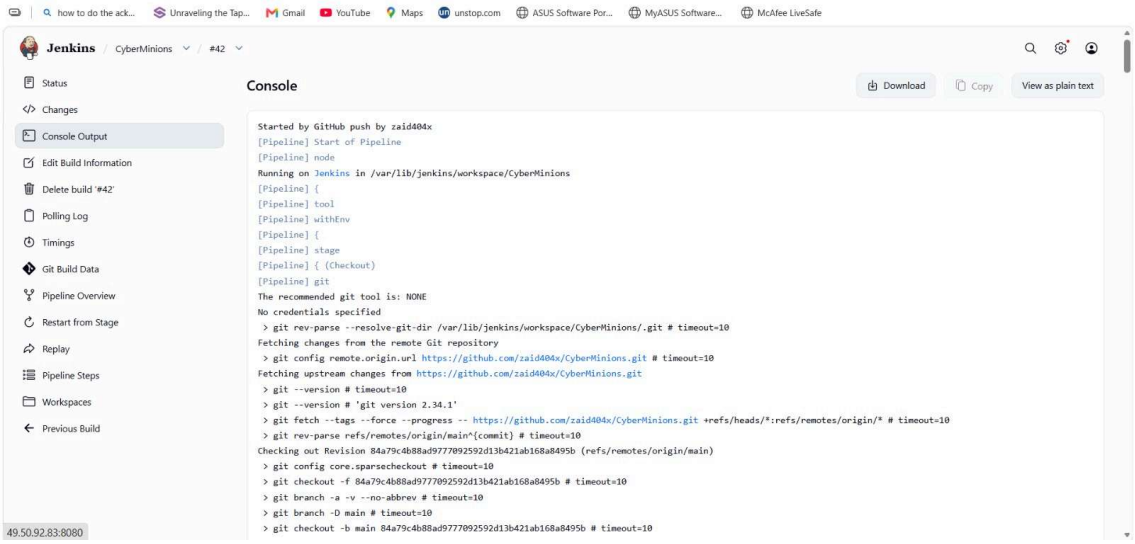
Stage 5: Trivy Image Scan
The built container image is scanned for security vulnerabilities.

Stage 6: Docker Push
Image is pushed to Docker Hub.
Example: docker push username/cyberminions:v1

Stage 7: Kubernetes Deployment
Application is deployed using:
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

Stage 8: Verification
Deployment status is verified using:
kubectl get pods
kubectl get svc



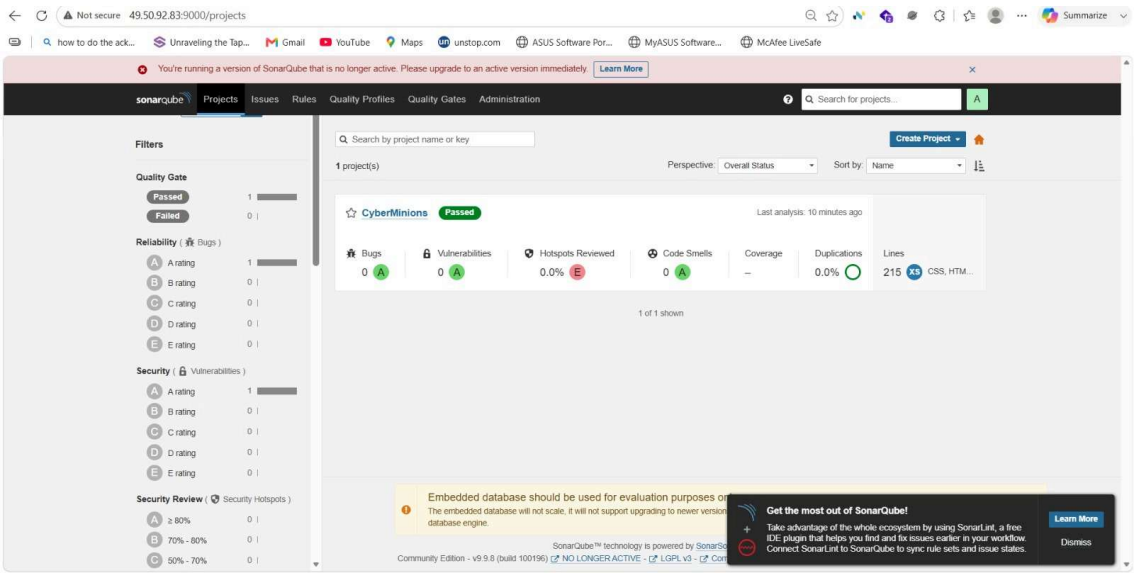


9. SonarQube Integration

SonarQube was integrated with Jenkins to perform automated code quality analysis.

Configuration Steps
SonarQube deployment. => Authentication token generation. => Jenkins plugin installation. => Jenkins credential configuration. => Pipeline integration.

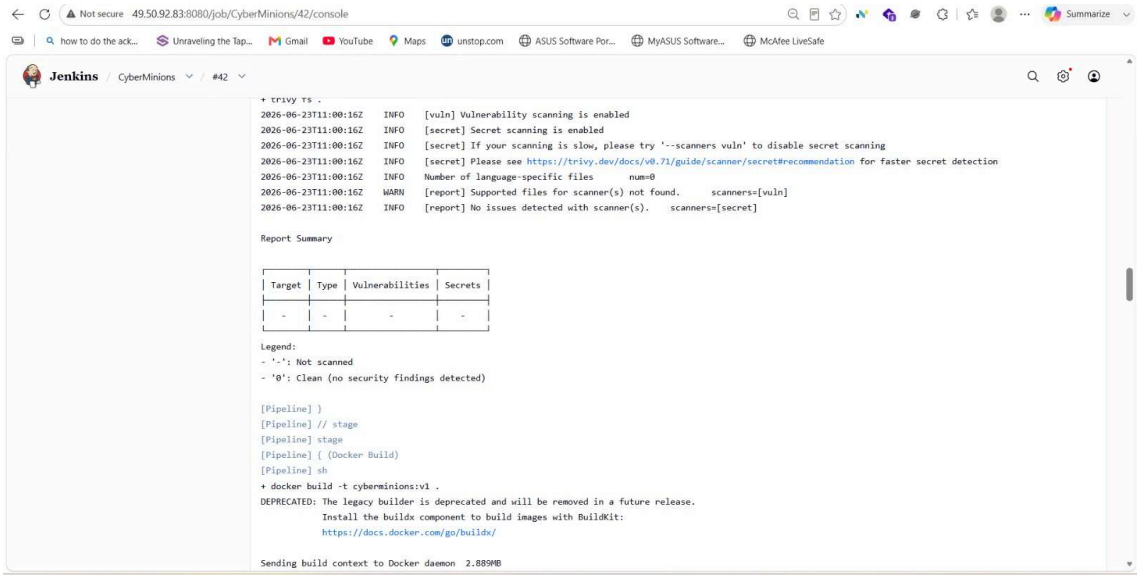
Outcome: Every build automatically performs source code inspection before deployment.



10. Trivy Security Scanning

Trivy was integrated into the CI/CD pipeline to implement DevSecOps security practices.
Filesystem Scan: trivy fs .
Container Image Scan: trivy image cyberminions:v1

Benefits: Early vulnerability detection, Container security validation, Security compliance improvement.




```
2026-06-23T11:00:17Z    INFO    [vuln] Vulnerability scanning is enabled
2026-06-23T11:00:17Z    INFO    [secret] Secret scanning is enabled
2026-06-23T11:00:17Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-06-23T11:00:17Z    INFO    [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
2026-06-23T11:00:18Z    INFO    Detected OS     family="alpine" version="3.23.4"
2026-06-23T11:00:18Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.23" repository="3.23" pkg_num=71
2026-06-23T11:00:18Z    INFO    Number of language-specific files       num=0
2026-06-23T11:00:18Z    WARN    Using severities from other vendors for some vulnerabilities. Read
https://trivy.dev/docs/v0.71/guide/scanner/vulnerability#severity-selection for details.

Report Summary

┌──────────────────────────────────┬────────┬─────────────────┬─────────┐
│              Target              │  Type  │ Vulnerabilities │ Secrets │
├──────────────────────────────────┼────────┼─────────────────┼─────────┤
│ cyberminions:v1 (alpine 3.23.4)  │ alpine │        2        │    -    │
└──────────────────────────────────┴────────┴─────────────────┴─────────┘

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)


For OSS Maintainers: VEX Notice
--------------------------------
If you're an OSS maintainer and Trivy has detected vulnerabilities in your project that you believe are not actually exploitable, consider issuing a VEX
(Vulnerability Exploitability eXchange) statement.
VEX allows you to communicate the actual status of vulnerabilities in your project, improving security transparency and reducing false positives for your users.
Learn more and start using VEX: https://trivy.dev/docs/v0.71/guide/supply-chain/vex/repo#publishing-vex-documents

To disable this notice, set the TRIVY_DISABLE_VEX_NOTICE environment variable.
```

```
cyberminions:v1 (alpine 3.23.4)
==============================
Total: 2 (UNKNOWN: 0, LOW: 1, MEDIUM: 0, HIGH: 1, CRITICAL: 0)

┌─────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────┐
│ Library │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                          Title                           │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
│ libexpat │ CVE-2026-45186 │ HIGH     │ fixed  │ 2.7.5-r0          │ 2.8.1-r0      │ libexpat: denial of service via crafted XML input        │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-45186               │
│         ├────────────────┼──────────┤        │                   │               ├──────────────────────────────────────────────────────────┤
│         │ CVE-2026-41080 │ LOW      │        │                   │               │ libexpat: expat: libexpat: Denial of Service via hash    │
│         │                │          │        │                   │               │ flooding with crafted XML...                             │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-41080               │
└─────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────┘
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Push)
[Pipeline] sh
+ docker tag cyberminions:v1 zaidpathan31/cyberminions:v1
[Pipeline] sh
+ docker push zaidpathan31/cyberminions:v1
The push refers to repository [docker.io/zaidpathan31/cyberminions]
696fe85c76cd: Waiting
6a0ac1617861: Waiting
9d80b52aca3c: Waiting
e0b71e755d67: Waiting
747ffc4966dd: Waiting
6ed38c8bc5c9: Waiting
f982d9d7587e: Waiting
```

11. Docker Implementation

Docker was used to package the application into portable containers.

Activities: Dockerfile creation, Image building, Local testing, Registry publishing.
Benefits: Portability, Consistency, Simplified deployment.

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Build) (show)
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Trivy Image Scan)
[Pipeline] sh
+ trivy image cyberminions:v1
2026-06-23T11:00:17Z    INFO    [vuln] Vulnerability scanning is enabled
2026-06-23T11:00:17Z    INFO    [secret] Secret scanning is enabled
2026-06-23T11:00:17Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-06-23T11:00:17Z    INFO    [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
2026-06-23T11:00:18Z    INFO    Detected OS     family="alpine" version="3.23.4"
2026-06-23T11:00:18Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.23" repository="3.23" pkg_num=71
2026-06-23T11:00:18Z    INFO    Number of language-specific files       num=0
2026-06-23T11:00:18Z    WARN    Using severities from other vendors for some vulnerabilities. Read
https://trivy.dev/docs/v0.71/guide/scanner/vulnerability#severity-selection for details.

Report Summary
```

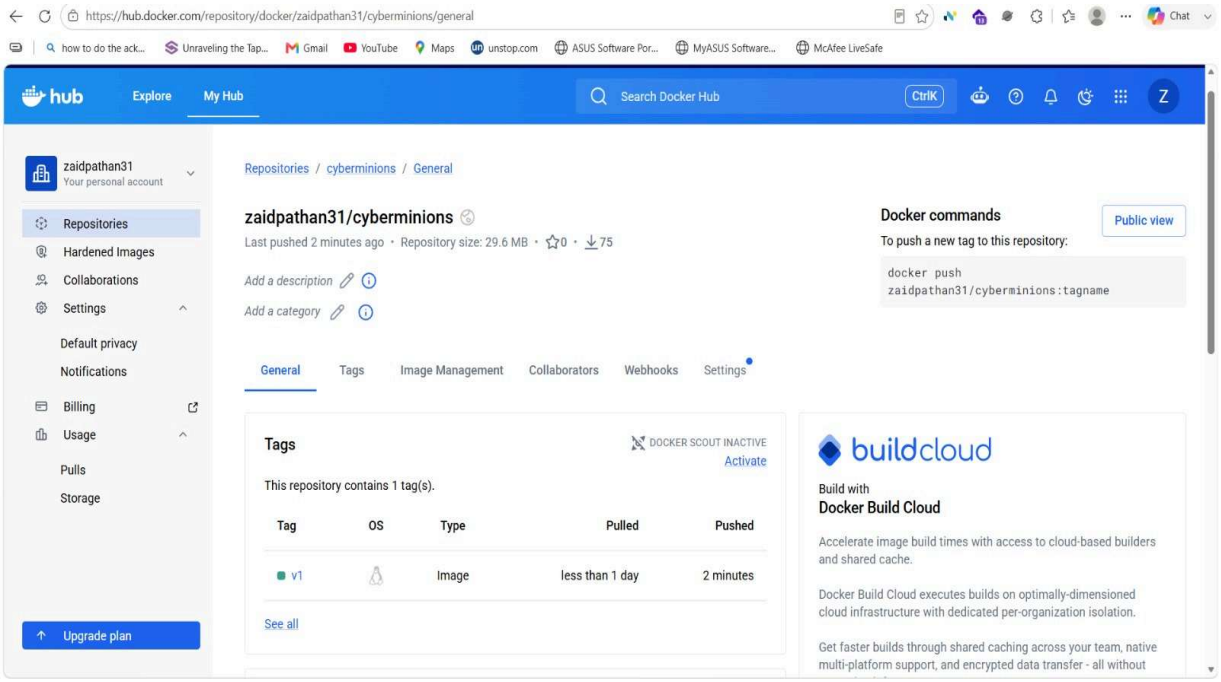
12. Docker Hub Integration

Docker Hub was used as the container registry.

Workflow

Docker Build
↓
Docker Push
↓
Docker Hub
↓
Kubernetes Pull

Benefits: Centralized image storage, Easy image distribution, Version management.



13. Kubernetes Deployment

Application deployment was managed using Kubernetes manifests.

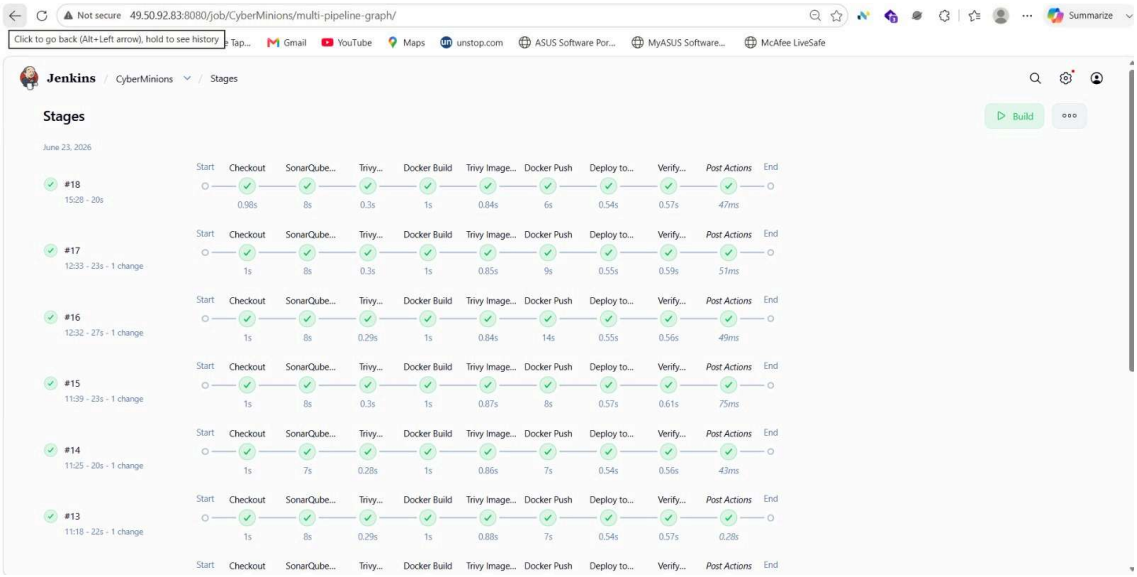
Deployment Resource
Responsibilities: Pod creation, Replica management, Rolling updates.
Service Resource
Responsibilities: Application exposure,Traffic routing, Verification Commands, kubectl get deployments, kubectl get pods, kubectl get svc.

14. GitHub Webhook Automation

GitHub Webhooks were configured to automate pipeline execution.

Workflow
Code Push
↓
GitHub Webhook
↓
Jenkins Build Trigger

Benefits: Fully automated CI/CD, Faster deployments, Reduced manual effort.

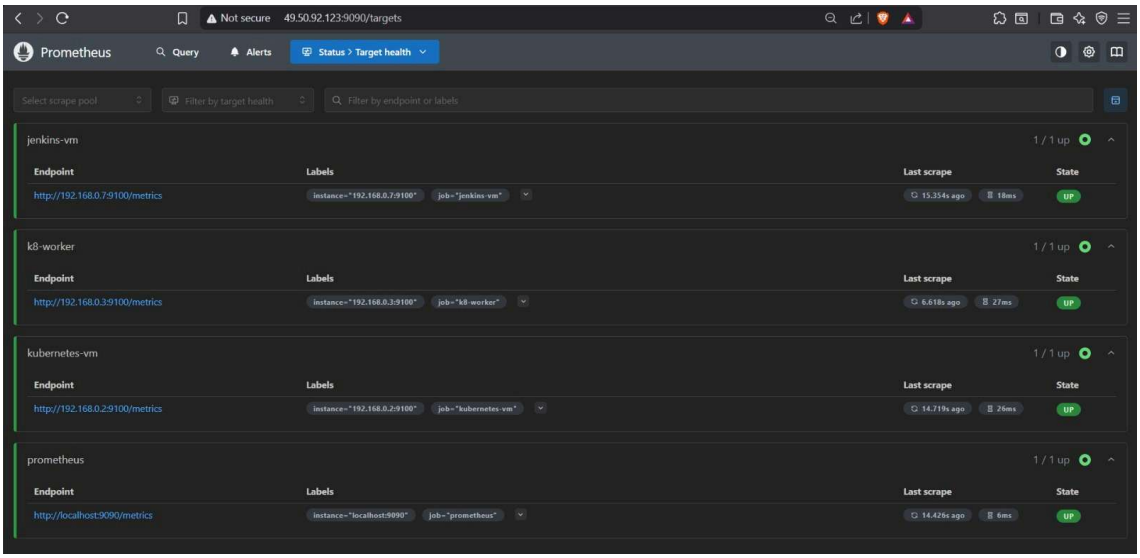


15. Monitoring Implementation

To achieve operational visibility, Prometheus and Grafana were deployed.

Prometheus: Prometheus was configured to collect metrics from:
Jenkins Server
Metrics monitored: CPU Usage, Memory Usage, Disk Utilization, System Load.
Kubernetes Cluster
Metrics monitored: Node Status,Pod Status, CPU Consumption, Memory Consumption, Cluster Health

Benefits: Real-time monitoring, Metric collection, Alerting capability.

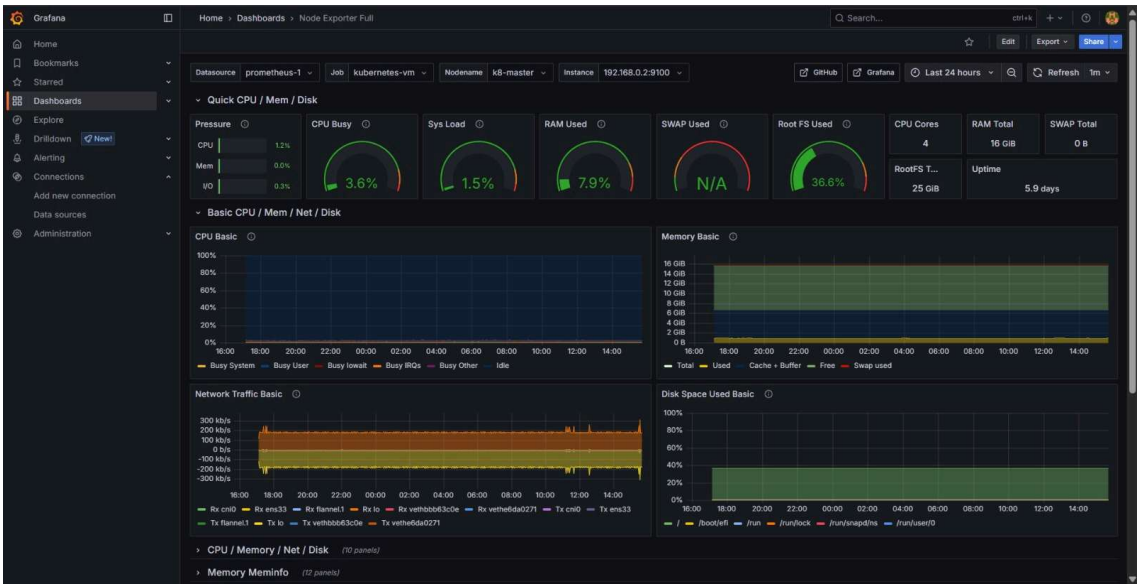


Grafana: Grafana was connected to Prometheus as the data source.
Dashboards Created

Jenkins Monitoring Dashboard

Displays:

- CPU utilization
- Memory utilization
- Disk usage
- System performance



Kubernetes Monitoring Dashboard

Displays:

- Node health
- Pod metrics
- CPU consumption
- Memory consumption
- Cluster status

Benefits: Centralized monitoring, Interactive visualization, Operational insights.

16. Results

The project successfully implemented:

✅ Automated CI/CD Pipeline

✅ Source Code Management

✅ Static Code Analysis

✅ Vulnerability Scanning

✅ Docker Containerization

✅ Docker Hub Integration

✅ Kubernetes Deployment

✅ Automated GitHub Webhook Triggering

✅ Infrastructure Monitoring

✅ Grafana Visualization Dashboards

17. Challenges Faced

- SonarQube Integration
- Plugin configuration issues.
- Authentication token configuration.
- Docker Hub Authentication
- Registry authentication failures.
- Credential management.
- Kubernetes Access
- kubeconfig configuration issues.
- Jenkins-to-Kubernetes authentication.
- Webhook Configuration
- Endpoint accessibility validation.
- Trigger verification

18. Future Enhancements

- OWASP Dependency Check Integration
- Slack Notifications
- Email Alerts
- ArgoCD GitOps Deployment
- Kubernetes Ingress Controller
- High Availability Kubernetes Cluster
- Centralized Logging using ELK Stack
- Runtime Security using Falco

19. Conclusion

This project successfully demonstrates the implementation of a complete DevSecOps pipeline using modern cloud-native technologies. The solution automates software delivery while integrating security and monitoring practices throughout the deployment lifecycle.

The integration of GitHub, Jenkins, SonarQube, Trivy, Docker, Docker Hub, Kubernetes, Prometheus, and Grafana provides a production-oriented workflow that follows industry-standard DevSecOps principles and improves deployment efficiency, security, scalability, and observability.